

Implement Client Script & UI Policy (Incident)

Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behavior and validation directly at the user interface level. Implementing standardized UI Policies and Client Scripts helps control field visibility, mandatory status, and save-time validation, ensuring that Incident records meet defined business rules before they are submitted.

Objective

The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met. The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform.

Skills

Incident Management
UI Policy
UI Policy Actions
Client Scripts
Form Validation

Task 1: Create UI Policy on Incident

Activity 1: Create UI Policy

1. Log in to the **ServiceNow instance** with administrative or configuration access.

2. Navigate to:
System UI → UI Policies
3. Click **New** to create a new UI Policy record.
4. Fill in the following details:
 - **Name:** High Impact Control
This name clearly represents the purpose of the policy, which is to apply additional controls when an Incident has a high impact.
 - **Table:** Incident
Specifies that the UI Policy applies to Incident records.
 - **Active:** true
Enables the UI Policy so it is enforced on the Incident form.
5. Configure the **Conditions** section as follows:
 - **Field:** Impact
 - **Operator:** is
 - **Value:** 1 – High
6. This condition ensures the UI Policy is triggered only when the Incident impact is set to High.
7. And Ui policy action Assignment group
8. Check mandatory
9. Set **Reverse if false** to **true**.
This ensures that when the condition is not met, any field changes applied by the policy are automatically reverted.
10. Review the configuration to ensure accuracy.
11. Click **Submit** to save the UI Policy.

dev221149.service-now.com/now/nav/ui/classic/params/target/sys_ui_policy_list.do%3Fsysparm_userpref_module%3Dbdcf56100a0a0b1b007a4abff76f34ef%26sysp...

servicenow All

system ui

UI Policies

UI Policies Updated Search

Actions on selected rows... New

Short description	Table	Conditions	Reverse if false	On lo
High Impact Control	Incident [incident]	impact=1^EQ	true	true
Hide for Daily Rotations	Roster [cmn_rota_roster]	rotation_interval_type=daily^EQ	true	true
Weekly Rotations Not All Day	Roster [cmn_rota_roster]	rotation_interval_type=weekly^rotation_a...	false	true
Weekly Rotations All Day	Roster [cmn_rota_roster]	rotation_interval_type=weekly^rotation_a...	false	true
No Reminders	Roster [cmn_rota_roster]	send_reminders=false^EQ	true	true
Show pricing implications checkbox	Question [question]	type=8^ORtype=21^EQ	true	true
Show price if checked	Question [question]	type=7^EQ	true	true
Duration only applies to request type categories	Dynamic category [sc_category_top_n]	type=sc_req_item^EQ	true	true

1 to 20 of 2 904

dev221149.service-now.com/now/nav/ui/classic/params/target/sys_ui_policy.do%3Fsys_id%3Dc346c375c32343105415382d0501319e%26sysparm_record_target%3...

servicenow All

system ui

UI Policy - High Impact Control

UI Policy High Impact Control

Update Delete

UI policies change fields on a form based on a set of conditions. Use UI policies to show or hide fields, or to make fields mandatory or read-only based on these conditions. A UI policy specifies one or more actions to take when the policy is triggered. First, create the policy. Then add as many actions as necessary. UI policy actions are applied only if all of the following conditions are met:

- The UI Policy is **Active**
- The items in the **Conditions** field evaluate to true
- The field specified in the UI policy action is present on the specified form

[More info](#)

Table: Incident [incident] Application: Global

Active ☒

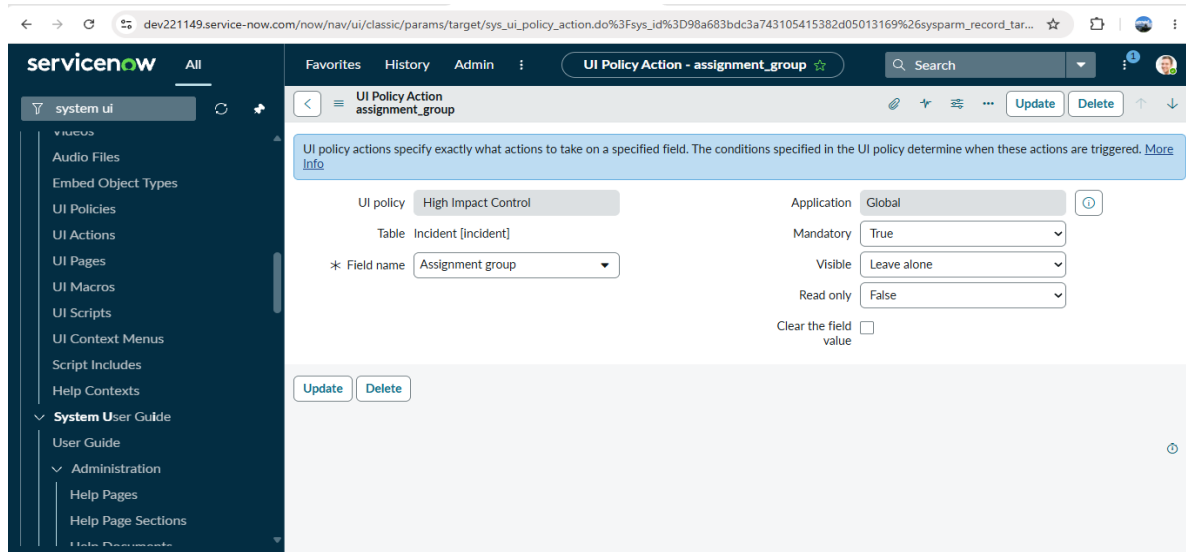
* Short description: High Impact Control

When to Apply

Conditions: Add Filter Condition Add OR Clause

Impact is 1 - High AND

Update Delete



Task 2: Create UI Policy Action – Urgency

Activity 1: Configure the Urgency Field

1. Open the UI Policy Actions

Go to the **High Impact Control** UI Policy you created earlier. Scroll down to the **UI Policy Actions** related list. This is where you define the specific behavior of fields when the policy conditions are met.

2. Create a New Action

Click **New** to create a UI Policy Action. This lets you configure how individual fields behave when the UI Policy is triggered.

3. Fill in the Action Details

- **Field name:** Urgency
Select the **Urgency** field on the Incident form. This is the field you want to control.
- **Read-only:** true
Check this option to make the Urgency field read-only whenever the UI Policy condition is met (Impact = High). Users will be able to see the field but cannot change its value.

- **Visible:** Leave this field as is (do not change).
By leaving it unchanged, the field will remain visible on the form, but its behavior will be controlled by the Read-only setting.

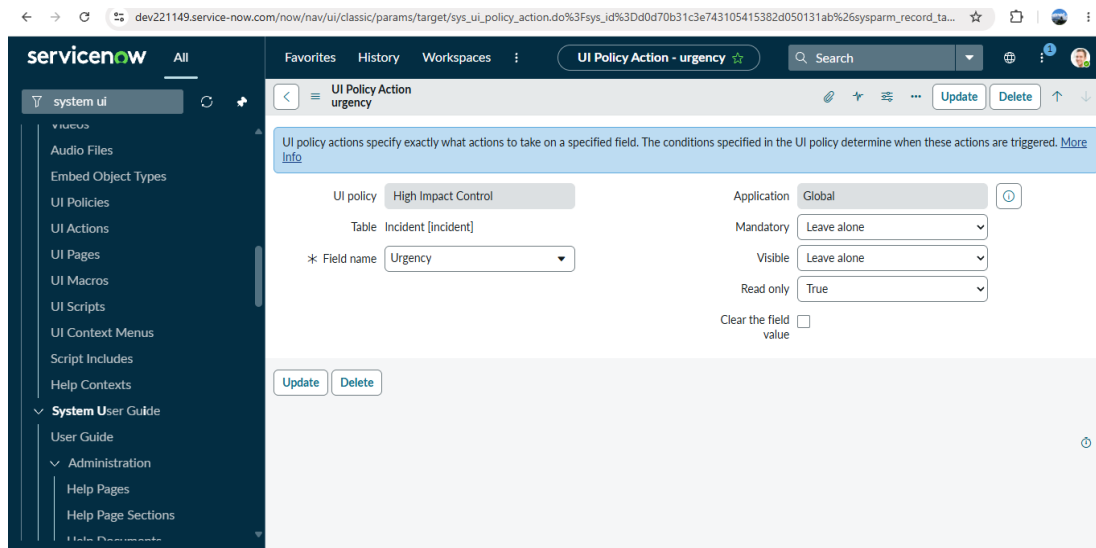
4. Submit the Action

Click **Submit** to save the UI Policy Action. The action is now linked to the High Impact Control UI Policy and will automatically apply to the Urgency field when an Incident has High Impact.

5. Optional Verification

To test the action:

- Open an Incident record.
- Set the **Impact** field to **High**.
- Check that the **Urgency** field becomes read-only.
- Change Impact to a different value to confirm that the field reverts back if needed.



Task 3: Create onChange Client Script

Activity 1: Create Client Script

1. **Navigate to Client Scripts:**
Go to System UI → Client Scripts.
2. **Click New** to create a new Client Script.
3. **Fill in the details:**
 - **Name:** Auto set urgency for high impact – clearly shows the purpose.
 - **Table:** Incident – the script runs only on Incident records.
 - **Type:** onChange – executes when a field value changes.
 - **Field name:** Impact – triggers the script when Impact is updated.
 - **Active:** true – ensures the script runs immediately.
4. **Paste the script** in the script editor section.
5. **Click Submit** to save the Client Script.
6. **Verify:** Open an Incident record and change the Impact field to **High**. The Urgency field should update automatically

Script:

```
function onChange(control, oldValue, newValue, isLoading) {  
  
    if (isLoading || newValue === "") {  
  
        return;  
  
    }  
  
    if (newValue === '1') {  
  
        g_form.setValue('urgency', '1');  
  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');  
    }  
}
```

}

5. Click Submit.

The screenshot shows the ServiceNow interface for managing Client Scripts. The left sidebar contains a navigation menu with 'Service Catalog', 'Catalog Administration', 'Catalog Client Scripts', and 'System Definition'. The main area displays a table of Client Scripts. The table has columns for Name, Active, Table, Application, View, Type, and Updated. The table lists several scripts, including 'Maximum Duration Range Check', 'Create default value', 'Show overlap field based on data labels', 'Set protocol based on auth type', 'Set default values for OIDC PKCE flow', 'Make notification read only', 'Deprecated - SN - Make serial number man', and 'Validate StatefulSet'.

Name	Active	Table	Application	View	Type	Updated
Maximum Duration Range Check	true	Transaction Quota Rule [sysrule_quota]	Global		onChange	2012-05-30 14:47:01
Create default value	true	Response Template Channel [sn_templated_snip_channel]	Templated Snippets		onChange	2020-09-22 17:06:27
Show overlap field based on data labels	true	Widgets [pa_widgets]	Global		onChange	2021-04-23 02:59:51
Set protocol based on auth type	false	HTTP(s) Connection [http_connection]	Global		onChange	2017-06-02 15:49:24
Set default values for OIDC PKCE flow	true	OAuth OIDC Entity [oauth_oidc_entity]	Global		onChange	2020-02-28 00:59:58
Make notification read only	true	Additional Notification Recipients [sys_notification_recipient]	Global		onLoad	2021-02-05 22:42:13
Deprecated - SN - Make serial number man	false	Asset [aim_asset]	Global		onChange	2025-03-06 00:42:54
Validate StatefulSet	true	MID KBs Deployment Request	Global		onChange	2022-10-09

The screenshot shows the configuration page for a Client Script named 'Auto set urgency for high impact'. The page includes a warning message about strict mode and disabled DOM access. The configuration fields are as follows:

- Name: Auto set urgency for high impact
- Table: Incident [incident]
- UI Type: Desktop
- Type: onChange
- Field name: Impact
- Application: Global
- Active: ☒
- Inherited: ☐
- Global: ☒

The Description and Messages fields are empty. The Script field contains the following code:

```
1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading || newValue == '') {
3     return;
4   }
5 }
```

Task 4: Create onSubmit Client Script (Save Validation)

Activity 1: Create Client Script

1. **Navigate to Client Scripts:**

Go to System UI → Client Scripts in your ServiceNow instance.

2. **Click New** to create a new Client Script.

3. **Fill in the details:**

- **Name:** Prevent save if Assigned To missing – clearly indicates that the script prevents saving if the field is empty.
- **Table:** Incident – applies the script only to Incident records.
- **Type:** onSubmit – executes when the user tries to save the record.
- **Active:** true – ensures the script is enforced immediately.

4. **Paste the script** into the script editor section. This script should check if the **Assigned To** field is empty and prevent the record from saving if it is.

5. **Click Submit** to save the Client Script.

6. **Verify:** Try saving an Incident without filling the **Assigned To** field. The script should prevent the save and display a warning.

7. Paste the script:

```
function onSubmit() {  
  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == "") {  
  
        g_form.showErrorBox(  
  
            'assigned_to',  
  
            'Assigned To is mandatory for High impact incidents.'  
  
        );  
    }  
}
```



```

    return false;

}

return true;

}

```

8. Click **Submit**.

dev221149.service-now.com/now/nav/ui/classic/params/target/sys_script_client.do%3Fsys_id%3D41fa8fb5c3e743105415382d0501318d%26sysparm_record_target%...

servicenow All

client scripts

FAVORITES
No Results

ALL RESULTS

- Service Catalog
 - Catalog Administration
 - Catalog **Client Scripts**
 - System Definition
 - Client Scripts**

Client Script - Prevent save if Assigned To missing

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name: Prevent save if Assigned To missing

Table: Incident [Incident]

UI Type: Desktop

Type: onSubmit

Application: Global

Active: ☒

Inherited: ☐

Global: ☒

Description:

Messages:

Script

```

1 function onSubmit() {
2   if (g_form.getValue('impact') == '1' &&
3     g_form.getValue('assigned_to') == '') {
4     g_form.showErrorMessage(

```

Task 5: Create onCellEdit Client Script

Activity 1: Create Client Script

- Navigate to Client Scripts:**
Go to System UI → Client Scripts in your ServiceNow instance.
- Click New** to create a new Client Script.
- Fill in the details:**

- **Name:** Prevent state change via list edit – clearly indicates that the script prevents users from changing the State field directly in a list.
 - **Table:** Incident – applies the script only to Incident records.
 - **Type:** onCellEdit – runs whenever a field is edited directly in a list view.
 - **Field name:** State – the script will monitor changes to the State field.
 - **Active:** true – ensures the script is enforced immediately.
4. **Paste the script** into the script editor section. The script should prevent unauthorized changes to the State field when editing from the list view.
 5. **Click Submit** to save the Client Script.
 6. **Verify:** Open an Incident list and try to change the State field directly in the list. The script should block the change as intended.

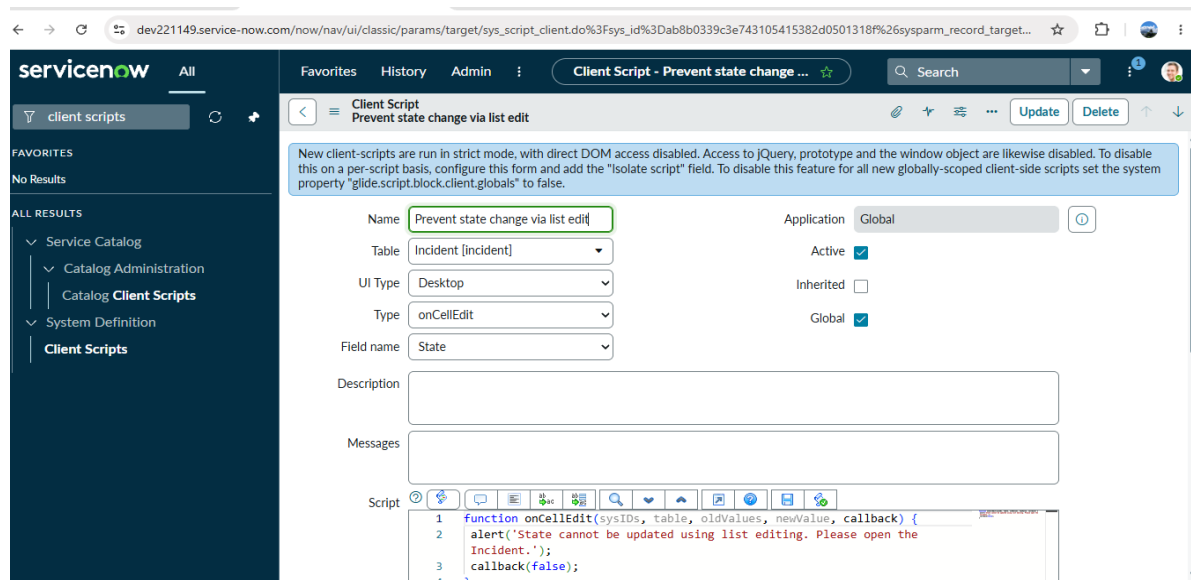
7. In the Script field, paste the following code:

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
  
    alert('State cannot be updated using list editing. Please open the Incident.');
```

callback(false);

```
}
```

8. Click **Submit**.



Task 6: Testing the Configuration

Activity 1: Test Mandatory Enforcement

- 1. Navigate to Incident:**
Go to Incident → Create New to open a new Incident form.
- 2. Set the Impact Field:**
Select **High** for the **Impact** field. This will trigger the UI Policy and Client Script behaviors you configured earlier.
- 3. Leave Assigned To Empty:**
Do not fill in the **Assigned To** field. This will test the onSubmit Client Script that prevents saving if Assigned To is missing.
- 4. Click Submit:**
Attempt to save the Incident record.
- 5. Verify the Result:**

- The Incident should **not be saved** because Assigned To is required when Impact is High.
- An **error message** should appear, indicating that the Assigned To field cannot be empty.
- Confirm that all UI Policy actions, such as read-only fields or automated urgency updates, are working as expected.

servicenow All

incident

FAVORITES No Results

ALL RESULTS

- Self-Service
- Incidents
- Watched Incidents
- Service Desk
- Incidents
- Incident
 - Create New
 - Assigned to me
 - Open
 - Open - Unassigned
 - Resolved
 - All
 - Overview

Incident - Create INC0010006

Incident New record

Number INC0010007

Channel -- None --

* Caller

State New

Category Inquiry / Help

Impact 3 - Low

Subcategory -- None --

Urgency 3 - Low

Service

Priority 5 - Planning

Service offering

Assignment group

Configuration item

Assigned to

* Short description

Description

Submit Resolve

Related Search Results

Notes Related Records Resolution Information

servicenow All

incident

FAVORITES No Results

ALL RESULTS

- Self-Service
- Incidents
- Watched Incidents
- Service Desk
- Incidents
- Incident
 - Create New
 - Assigned to me
 - Open
 - Open - Unassigned
 - Resolved
 - All
 - Overview

Incident - Create INC0010007

Incident New record

Urgency set to High for High impact incident.

Number INC0010007

Channel -- None --

* Caller Abraham Lincoln

State New

Category Inquiry / Help

Impact 1 - High

Subcategory -- None --

Urgency 1 - High

Service

Priority 1 - Critical

Service offering

* Assignment group CAB Approval

Configuration item

Assigned to

* Short description cab approve

Description

Submit Resolve

Assigned To is mandatory for High Impact Incidents.

Activity 2: Test Successful Save

1. Fill the Assigned To Field:

Open the Incident form and select a user in the **Assigned To** field. This ensures the onSubmit Client Script condition is satisfied.

2. Click Submit:

Attempt to save the Incident record.

3. Verify the Result:

- The Incident should be **saved successfully** without any errors.
- Confirm that all other configured behaviors, such as:
 - **Urgency auto-setting** (from the onChange Client Script)
 - **Read-only fields** (from the UI Policy Action)
 - **State restrictions** (from the onCellEdit Client Script) are functioning as expected

The screenshot displays the ServiceNow 'Incident - Create INC0010007' form. The left sidebar shows the navigation menu with 'Incident' selected. The main form area contains the following fields and values:

- Number: INC0010007
- Channel: -- None --
- * Caller: Abraham Lincoln
- State: New
- Category: Inquiry / Help
- Impact: 1 - High
- Subcategory: -- None --
- Urgency: 1 - High
- Service: (empty)
- Priority: 1 - Critical
- Service offering: (empty)
- * Assignment group: CAB Approval
- Configuration item: (empty)
- Assigned to: Luke Wilson
- * Short description: cab approve
- Description: (empty text area)

At the top of the form, a message states: 'Urgency set to High for High Impact Incident.' The bottom of the form includes a 'Related Search Results' button.

Activity 3: Reverse Condition Test

1. Open an Existing Incident:

Go to Incident → Open and select an Incident where **Impact** is currently set to **High**.

2. Change Impact:

Update the **Impact** field from **High** to **Medium**. This will test the **Reverse if false** functionality of your UI Policy and the behavior of your Client Scripts.

3. Verify the Field Behavior:

- **Assigned To** should no longer be mandatory.
- **Urgency** should now be editable, meaning the read-only UI Policy action is reversed.

4. Save the Record:

Click **Submit** to save the changes. The record should save successfully without errors, confirming that the reverse condition works as intended.

The screenshot displays the ServiceNow Incident form for record INC0010007. The form is divided into several sections. The top section contains the incident number and title. Below this, there are fields for Caller (Abraham Lincoln), Category (Inquiry / Help), Subcategory (None), Service, Service offering, and Configuration item. To the right of these fields are dropdown menus for Channel (None), State (In Progress), Impact (2 - Medium), Urgency (1 - High), and Priority (2 - High). The Assignment group is set to CAB Approval, and the Assigned to field is empty. The Short description is 'cab approve', and the Description field is empty. At the bottom of the form, there are tabs for Notes, Related Records, and Resolution Information. The left sidebar shows the navigation menu with 'Incident' selected.

Activity 4: Test List Edit Blocking

1. Navigate to the Incident List:

Go to Incident → All to view all Incident records in a list view.

2. Attempt a List Edit:

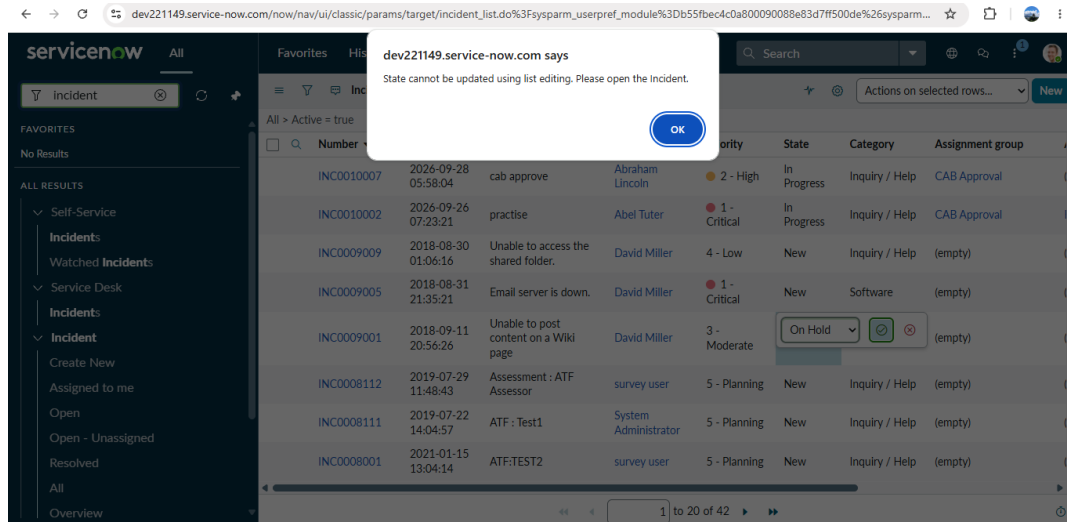
- Locate any Incident in the list.
- **Double-click the State field** to try editing it directly from the list.

3. Verify the Behavior:

- An **alert message** should appear, indicating that direct changes to the State field are not allowed.
- The **State value should remain unchanged**, confirming that the onCellEdit Client Script is working correctly.

The screenshot displays the ServiceNow incident list interface. The left sidebar shows the navigation menu with 'Incident' selected. The main table lists incidents with the following columns: Number, Opened, Short description, Caller, Priority, State, Category, and Assignment group. The 'State' column is highlighted, and a double-click action is being performed on the 'In Progress' state of incident INC0010007. The table contains the following data:

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group
INC0010007	2026-09-28 05:58:04	cab approve	Abraham Lincoln	2 - High	In Progress	Inquiry / Help	CAB Approval
INC0010002	2026-09-26 07:23:21	practise	Abel Tuter	1 - Critical	In Progress	Inquiry / Help	CAB Approval
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry / Help	(empty)
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)
INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)
INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)
INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)
INC0007002	2018-10-16 23:47:54	Need access to the common drive	David Miller	4 - Low	New	Inquiry / Help	(empty)



Activity 5: Test Form-Based Update

1. Open the Incident Record:

Navigate to the same Incident record you used in previous tests.

2. Change the State:

On the **Incident form**, update the **State** field to a new value. This tests that changes via the form (not the list) are allowed.

3. Click Update:

Save the record by clicking **Update**.

4. Verify the Result:

- The **State** change should be saved successfully.
- Confirm that other fields, such as **Assigned To** and **Urgency**, still follow the configured UI Policy and Client Script rules.

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group
INC0010007	2026-09-28 05:58:04	cab approve	Abraham Lincoln	2 - High	New	Inquiry / Help	CAB Approval
INC0010002	2026-09-26 07:23:21	practise	Abel Tuter	1 - Critical	In Progress	Inquiry / Help	CAB Approval
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry / Help	(empty)
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)
INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)
INC0008111	2019-07-22 14:04:57	ATF : Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)
INC0008001	2021-01-15 13:04:14	ATF:TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)

Conclusion

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how **UI Policies** and **Client Scripts** can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using `onChange`, `onSubmit`, and `onCellEdit` scripts alongside UI Policy actions, the project ensures fields like **Assigned To** and **Urgency** respond appropriately to changes in **Impact**, maintaining clean and

consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.